Chronos Multicore CPU Scheduling Simulator

Bereket Wolderufael

CSE4300

Introduction

Core Features

- Simulates real CPU scheduling
- Supports multiple CPU cores
- Generates random workloads
- Produces CSV output + visualizations

```
(env) bereketgwol@Mac Chronos % ./schedsim --help
Usage: schedsim [OPTIONS]
Options:
  --algo, -a <ALGO>       Scheduling algorithm (FCFS, SJF, Priority, RR)
  --cores, -c <NUM>       Number of CPU cores (positive integer)
  --jobs, -j <NUM>        Number of jobs (positive integer)
  --quantum, -q <NUM>     Time quantum for Round Robin (positive integer, optional)
  --compare-all           Run all algorithms and compare results
  --help, -h              Show this help message
```

Scheduling Algorithms

Algorithms Implemented

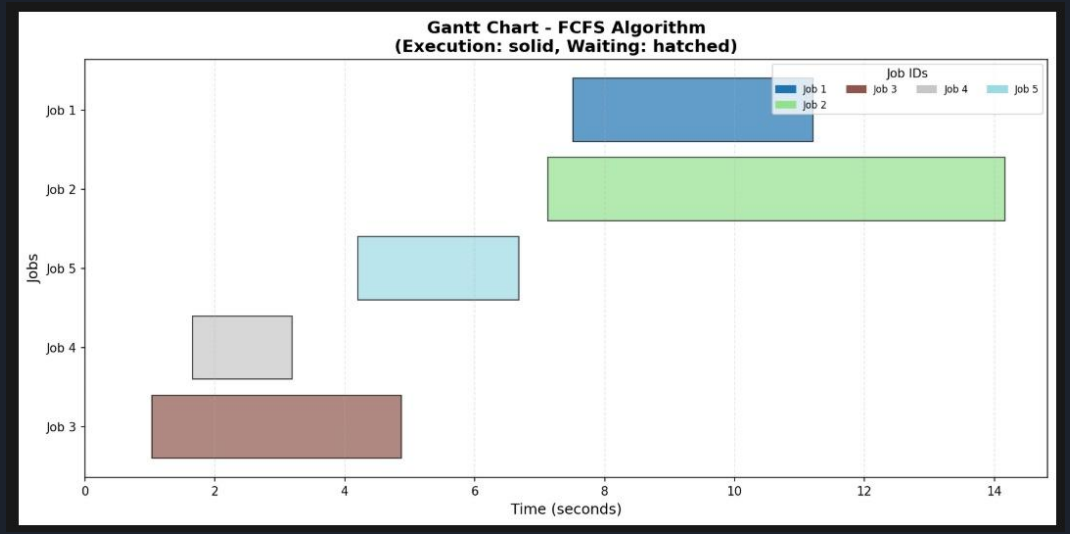
- FCFS
- SJF
- Priority
- Round Robin (preemptive)

```
Running FCFS...
Algorithm: FCFS
──────────────────────────────────────────────────────────
Job | Arrival | Burst | Start | Finish | Wait | Turnaround
──────────────────────────────────────────────────────────
  1 |     2.4 |   6.5 |   2.4 |    8.9 |  0.0 |       6.5
  2 |     4.0 |   9.2 |   8.9 |   18.0 |  4.9 |      14.0
  3 |     7.2 |   7.9 |  12.4 |   20.2 |  5.2 |      13.1
  4 |     2.7 |   9.6 |   2.7 |   12.4 |  0.0 |       9.6
  5 |     8.8 |   8.2 |  18.0 |   26.2 |  9.3 |      17.4
──────────────────────────────────────────────────────────
Average Waiting Time: 3.87
Average Turnaround Time: 12.13
CPU Utilization: 88.14%
Context Switches: 5
```

Multicore & Concurrency

Multicore Execution

- One worker thread per CPU core
- Independent local clocks
- Shared ready queue (mutex + condvar)
- Atomic context-switch counter



Metrics Collected

Metrics

- Waiting time
- Turnaround time
- Makespan
- CPU utilization
- Context switches

metrics.csv

```
algorithm,job_id,arrival_time,burst_time,priority,start_time,finish_time,waiting_time,turnaround_time,remaining_time
FCFS,4,1.66,1.53,4,1.66,3.19,0.00,1.53,0.00
FCFS,3,1.03,3.84,3,1.03,4.87,0.00,3.84,0.00
FCFS,5,4.20,2.48,5,4.20,6.68,0.00,2.48,0.00
FCFS,1,7.51,3.70,1,7.51,11.21,0.00,3.70,0.00
FCFS,2,7.13,7.03,1,7.13,14.16,0.00,7.03,0.00
```

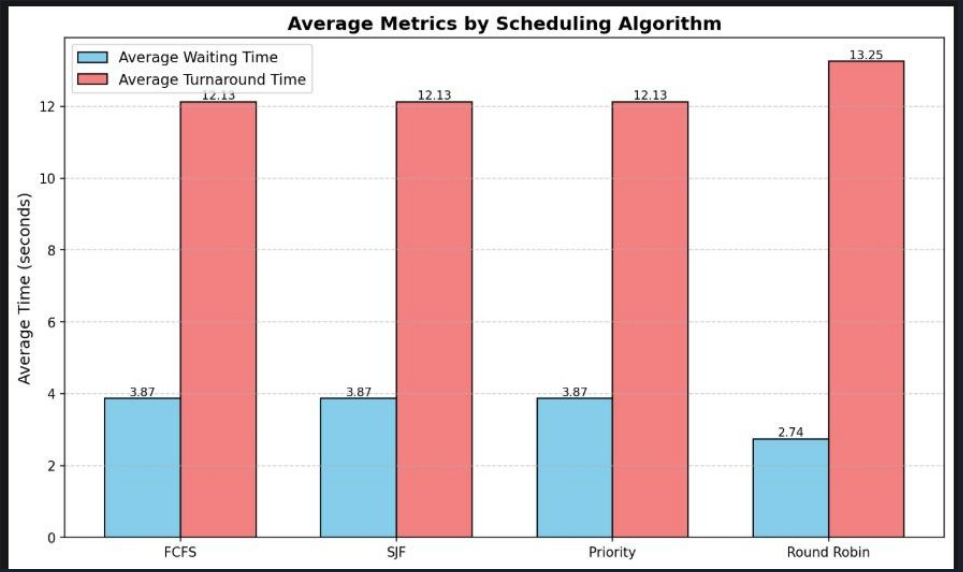
summary.csv

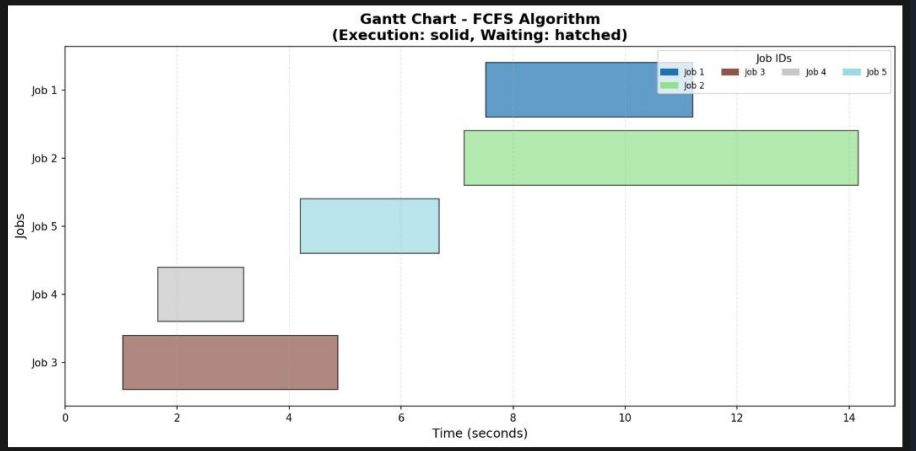
```
algorithm,avg_waiting_time,avg_turnaround_time,cpu_utilization,context_switches,num_jobs,makespan
FCFS,3.87,12.13,88.14,3,5,23.44
SJF,3.87,12.13,88.14,3,5,23.44
Priority,3.87,12.13,88.14,3,5,23.44
Round Robin,2.74,13.25,95.84,42,5,21.56
```

Visualization

Visualization Features

- Gantt chart
- Average metrics chart
- Utilization chart





CLI Usage

Command-Line Features

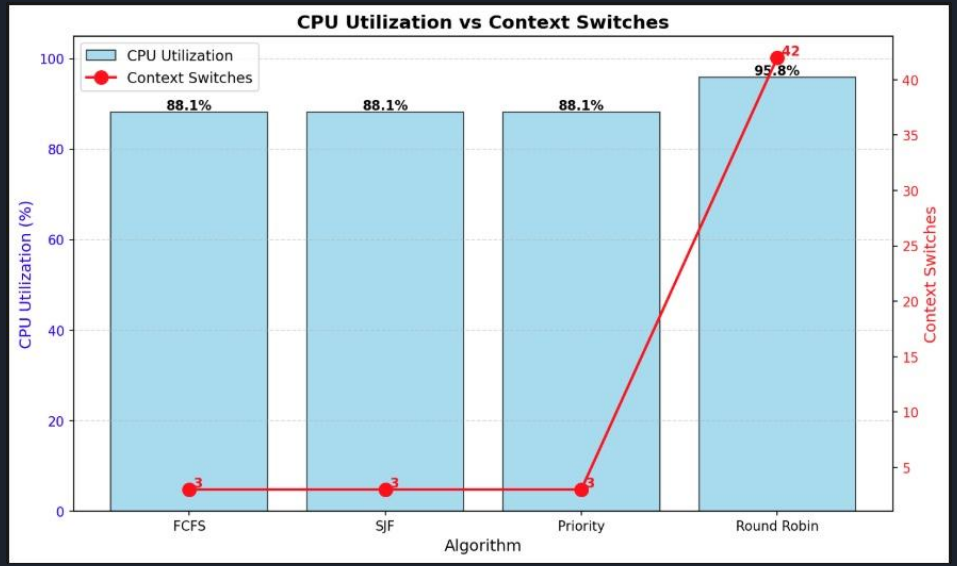
- --algo
- --cores
- --jobs
- --quantum
- --compare-all

```
(env) bereketgwol@Mac Chronos % ./schedsim --help
Usage: schedsim [OPTIONS]
Options:
  --algo, -a <ALGO>       Scheduling algorithm (FCFS, SJF, Priority, RR)
  --cores, -c <NUM>       Number of CPU cores (positive integer)
  --jobs, -j <NUM>        Number of jobs (positive integer)
  --quantum, -q <NUM>     Time quantum for Round Robin (positive integer, optional)
  --compare-all           Run all algorithms and compare results
  --help, -h              Show this help message
```

Compare-All Mode

Compare-All

- Runs FCFS, SJF, Priority, RR
- Default RR quantum = 2
- Only summary.csv generated
- Creates avg_metrics + utilization charts



Design Decisions

- Independent local time per core
- Random workload ranges
- CSV separation design
- Scheduling policy interface

```cpp
// Defines the interface that all scheduling algorithms must implement (Abstract base class)
class ISchedulingPolicy {
public:
    virtual ~ISchedulingPolicy() = default;


    // Returns: pointer to the selected job, or nullptr if no job is available
    // Note: The policy should not remove the job from the queue; that's the scheduler's responsibility
    virtual Job* getNextJob(std::vector<Job>& ready_queue) = 0;


    // current_time: current simulation time
    // Note: For preemptive algorithms (like RR), this may need to re-queue the job
    virtual void onJobCompletion(Job* completed_job, float current_time) = 0;

    // Get the name of the scheduling policy (for logging/output)
    virtual std::string getName() const = 0;

    // Check if the policy is preemptive
    // Preemptive policies can interrupt running jobs
    virtual bool isPreemptive() const = 0;

    // Optional time slice (seconds). Negative ⇒ run job to completion.
    virtual float getTimeSlice() const { return -1.0f; }
};
```

Possible Extensions

- I/O-bound job simulation
- Trace file input
- Preemptive SJF (SRTF)
- Priority aging
- Interactive visual dashboard

THANK YOU!!!